

Binary Search Templates

Binary search appears in two forms: searching an **array index**, or searching an **answer value**.

Master Quick Reference

#	Name	Description	Intuition	Time	Space	Key Implementation
704	Binary Search	Given a sorted array and a target, return its index. Return -1 if not found.	Both bounds converge to a single index <code>i</code> . With <code>lowerBound</code> , <code>i</code> is the first slot <code>>= target</code> , so check <code>nums[i]</code> . With <code>upperBound</code> , <code>i</code> is one past the last <code><= target</code> , so check <code>nums[i-1]</code> . Same loop, mirror edge.	$O(\log n)$	$O(1)$	Standard
34	Find First and Last Position of Element in Sorted Array	Return <code>[first, last]</code> index of target. Return <code>{-1, -1}</code> if absent.	Find the boundary where " <code>< target</code> " flips to " <code>>= target</code> " (first position), and the next boundary just past target.	$O(\log n)$	$O(1)$	Standard
35	Search Insert Position	Return the index where target is or would be inserted to keep the array sorted.	The insert position is the first index whose value is $\geq$ target — exactly <code>lowerBound</code> .	$O(\log n)$	$O(1)$	Standard
33	Search in Rotated Sorted Array	Array was sorted then rotated at an unknown pivot. Find target index, or -1.	First locate the rotation point with a boundary search, then the array splits into one sorted segment that must contain target — run a plain <code>lowerBound</code> there.	$O(\log n)$	$O(1)$	Standard
162	Find Peak Element	Return any index where <code>arr[k] > arr[k-1]</code> and <code>arr[k] > arr[k+1]</code> . Edges are $-\infty$.	Always walk uphill — an upward slope guarantees a peak to the right, so the slope direction is the "condition".	$O(\log n)$	$O(1)$	Standard
875	Koko Eating Bananas	Minimum eating speed so Koko finishes all piles in <code>h</code> hours (one pile per hour, at most <code>speed</code> bananas eaten).	Feasibility is monotonic in speed (faster always works), so binary search the speed axis for the smallest that finishes in time.	$O(n \log(\max(\text{piles})))$	$O(1)$	Standard
1011	Capacity to Ship Packages Within D Days	Minimum ship capacity to deliver all packages (in order) within <code>d</code> days.	Bigger capacity is always feasible, so binary search capacity for the smallest that ships within the day budget.	$O(n \log(\sum(\text{weights})))$	$O(1)$	Standard
410	Split Array Largest Sum	Split array into <code>m</code> non-empty subarrays to minimize the largest subarray sum.	A larger allowed max-sum needs fewer parts, so binary search that cap for the smallest splittable into $\leq m$ parts.	$O(n \log(\sum(\text{nums})))$	$O(1)$	Standard
1283	Find the Smallest Divisor Given a Threshold	Smallest positive divisor such that <code>sum of ceil(nums[i] / divisor) <= threshold</code> .	A bigger divisor shrinks the sum, so the condition flips false→true; binary search for the smallest divisor that fits the threshold.	$O(n \log(\max(\text{nums})))$	$O(1)$	Standard
1231	Divide Chocolate ← MAXIMIZE (the one that differs)	Cut chocolate into <code>k+1</code> pieces; keep the minimum-sweetness piece. Maximize that minimum.	A larger target-minimum yields fewer pieces, so feasibility flips true→false. MAXIMIZE = search the FIRST INFEASIBLE minimum and step back one: <code>condition(k) = !canDivide(k)</code> (<code>pieces < k+1</code>), false...false TRUE...true. Half-open	$O(n \log(\sum(s)))$	$O(1)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			<code>[lo, hi+1)</code> , return <code>i - 1</code> — no ceiling mid needed.			
4	Median of Two Sorted Arrays	Find the median of two sorted arrays of sizes m and n in $O(\log(m+n))$ time.	Pick a split of the smaller array; the rest of the split is forced. Shrink the partition until the left half's $\max \leq$ the right half's $\min$, then the median falls out of the boundary values.	$O(\log(\min(m, n)))$	$O(1)$	search smaller array
69	Sqrt(x)	Compute the integer square root of a non-negative integer x without using <code>sqrt()</code> .	Feasibility $k*k \leq x$ is true...true then false as k grows. MAXIMIZE = find the FIRST INFEASIBLE k with <code>condition(k) = k*k > x</code> (false...false TRUE...true), over <code>[1, x+1)</code> , and return <code>i - 1</code> — the largest k still feasible. No ceiling mid needed.	$O(\log x)$	$O(1)$	Standard
74	Search a 2D Matrix	Search for a target in an $m \times n$ matrix where each row is sorted and each row's first element $>$ previous row's last.	The layout means the whole matrix reads as a single sorted sequence; map a flat index to <code>(row, col)</code> and run a standard <code>[i, j)</code> lowerBound, then verify the landed value.	$O(\log(m*n))$	$O(1)$	flatten index to 2D
81	Search in Rotated Sorted Array II	Search in a sorted, rotated array that may contain duplicates.	Duplicates can make both ends equal to the midpoint so neither half looks sorted; in that ambiguous case shrink both bounds by one and retry.	$O(\log n)$ average, $O(n)$ worst	$O(1)$	ambiguous duplicates
153	Find Minimum in Rotated Sorted Array	Find the minimum element in a sorted, rotated array with no duplicates.	The minimum is the single point where the order breaks; if <code>arr[k] > arr[j]</code> the break is to the right, otherwise the min is at k or left.	$O(\log n)$	$O(1)$	Standard
240	Search a 2D Matrix II	Search for a target in an $m \times n$ matrix where rows and columns are both sorted.	From the top-right, moving left strictly decreases and moving down strictly increases, so each comparison eliminates a full row or column.	$O(m + n)$	$O(1)$	start top-right corner
278	First Bad Version	Find the first bad version using a provided <code>isBadVersion(n)</code> API. Minimize API calls.	Versions are good...good then bad...bad once corrupted; this is the classic first-true boundary search.	$O(\log n)$	$O(1)$	Standard
374	Guess Number Higher or Lower	Guess the number between 1 and n using a provided <code>guess()</code> API. Minimize calls.	<code>guess(k)</code> returns <code>-1</code> when the pick is lower than k , <code>1</code> when higher, <code>0</code> on a hit. So <code>guess(k) <= 0</code> is false...false TRUE...true as k grows, and the first true k is the answer.	$O(\log n)$	$O(1)$	Standard
475	Heaters	Given house and heater positions, find the minimum heater radius so every house is covered by some heater.	Each house needs the closest heater; the required radius is the largest of those closest distances. Find each house's nearest heater by binary search.	$O((m + n) \log m)$	$O(1)$	also check left neighbor
540	Single Element in a Sorted Array	Find the single non-duplicate element in a sorted array where all others appear twice. $O(\log n)$.	Before the unique element each pair starts at an even index; after it, the parity shifts. Force the midpoint even and check whether its partner matches to pick a half.	$O(\log n)$	$O(1)$	force k even
658	Find K Closest Elements	Find k integers in a sorted array closest to x , ordered by closeness then value.	The answer is a contiguous window of size k ; search for its start by comparing the distances of the two window edges to x and sliding toward the closer side.	$O(\log(n - k) + k)$	$O(k)$	search window start <code>[i, j]</code>

#	Name	Description	Intuition	Time	Space	Key Implementation
719	Find K-th Smallest Pair Distance	Find the k-th smallest absolute distance among all pairs in the array.	The count of pairs with distance $\leq d$ is monotonic in d , so binary search the distance axis; count pairs $\leq d$ with a sliding window over the sorted array.	$O(n \log n + n \log(\maxDist))$	$O(1)$	search distance value $[i, j]$
852	Peak Index in a Mountain Array	Find the peak index in a mountain array (element greater than both neighbors).	An upward slope means the peak is strictly to the right; a downward slope means the peak is here or to the left.	$O(\log n)$	$O(1)$	Standard
1060	Missing Element in Sorted Array	Find the kth missing number in a sorted array.	Missing count at each index increases monotonically, so binary search for the first index whose missing count is $\geq k$, then offset from the previous element.	$O(\log n)$	$O(1)$	kth missing beyond array end
1428	Leftmost Column with at Least a One	Find the leftmost column with at least one '1' in a binary matrix (rows sorted, accessed via BinaryMatrix API).	Each row is sorted 0...0 1...1; starting top-right, move left on a 1 (column might be even further left) and down on a 0, tracking the leftmost 1 seen.	$O(\text{rows} + \text{cols})$	$O(1)$	start top-right corner
1482	Minimum Number of Days to Make m Bouquets	Find the minimum number of days to make m bouquets of k adjacent flowers.	Waiting more days only adds bloomed flowers, so feasibility is monotonic; binary search the day axis and greedily count adjacent runs of bloomed flowers.	$O(n \log(\max(\text{bloomDay})))$	$O(1)$	impossible if not enough flowers
1539	Kth Missing Positive Number	Find the kth missing positive integer.	At each index the number of missing positives so far is <code>arr[k] - (k+1)</code> ; binary search the first index whose missing count is $\geq k$, then offset.	$O(\log n)$	$O(1)$	i missing-free slots + k
1818	Minimum Absolute Sum Difference	Find the minimum absolute sum difference after one substitution.	Total cost is the sum of absolute differences; one swap can only help where the closest available <code>nums1</code> value beats the current difference. Find that closest value by binary search and track the best gain.	$O(n \log n)$	$O(n)$	also check left neighbor
1891	Cutting Ribbons	Find the maximum ribbon length such that m ribbons of that length can be cut.	Longer pieces produce fewer ribbons, so feasibility flips <code>true</code> → <code>false</code> as length grows. Search the first INFEASIBLE length (<code>condition(k) = countRibbons(k) < k</code> , <code>false</code> ... <code>false</code> <code>TRUE</code> ... <code>true</code>) and step back one; that is the largest feasible length. If no length is feasible the loop lands on <code>lo</code> , giving <code>i - 1 = 0</code> .	$O(n \log(\max(\text{ribbons})))$	$O(1)$	$[1, \max+1)$ over lengths

* #162 uses `j = n-1` (not `n`) — loop reads `arr[k+1]`, out of bounds if `j = n`.

The Only Template — `[i, j)`, find first k with `condition(k)`

There is one binary search. Define a monotone `condition(k)` (`false`...`false` `TRUE`...`true`); the loop returns the **first k where `condition(k)` is true**. Then look at `i`: if `i == n` no index qualified, otherwise `i` is the boundary — check the value to decide.

Variables: `[i, j)` = half-open search range (answer lives in here) · `k` = midpoint · `condition(k)` = monotone test, `false`...`false` then `TRUE`...`true`; answer is the first `k` it holds **Pseudocode:**

```

i = 0, j = length      # half-open [i, j)
while range non-empty (i < j):
    k = midpoint of i and j
    if condition(k) is false: answer is to the right, set i = k+1
    else: k might be the answer, set j = k
# i is the first index satisfying condition (or == length if none)

```

```

int i = 0, j = nums.length;      // [i, j)
// find first k that meets condition(k)
while (i < j) {
    int k = i + (j - i) / 2;
    if (!condition(k)) {          // condition false → answer is strictly to the right
        i = k + 1;
    } else {                      // condition true → k might be the answer
        j = k;
    }
}
// i in [0, nums.length]; if i == nums.length, no k met condition(k)

```

Pick `condition(k)` :

Want	<code>condition(k)</code>	Answer after loop
<code>lowerBound</code> — first <code>>= target</code>	<code>nums[k] >= target</code>	<code>i</code>
<code>upperBound</code> — first <code>> target</code>	<code>nums[k] > target</code>	<code>i</code>
exact search (#704)	<code>nums[k] >= target</code>	<code>i < n && nums[i] == target ? i : -1</code>
insert position (#35)	<code>nums[k] >= target</code>	<code>i</code>
first / last occurrence (#34)	<code>>=</code> then <code>></code>	<code>lowerBound</code> , <code>upperBound - 1</code>
count of target	—	<code>upperBound - lowerBound</code>
minimize feasible X	<code>feasible(k)</code> over value range	<code>i</code>
maximize feasible X	<code>!feasible(k)</code> (first infeasible)	<code>i - 1</code>

The rule: the answer is always `i` . Guard `i == nums.length` (nothing met the condition) before reading `nums[i]` , then confirm the value.

How to Choose

Signal	<code>condition(k)</code>
Sorted array, find exact value	<code>arr[k] >= target</code> , then check <code>arr[i] == target</code>
Insert position / first occurrence	<code>arr[k] >= target</code> → return <code>i</code> (= <code>lowerBound</code>)
Last occurrence / count	<code>arr[k] > target</code> (= <code>upperBound</code>) ; <code>last = upperBound - 1</code> , <code>count = upper - lower</code>
Rotated / peak slope search	<code>condition(k)</code> compares <code>arr[k]</code> to a neighbor or endpoint
"minimum X such that <code>feasible(X)</code> "	<code>condition(k) = feasible(k)</code> over value range → return <code>i</code>
"maximum X such that <code>feasible(X)</code> "	<code>condition(k) = !feasible(k)</code> over <code>[lo, hi+1)</code> → return <code>i - 1</code>

The one rule: define `condition(k)` so it is monotone `false...false TRUE...true` , then the answer is the first `k` that flips it true (`j = k` on true, `i = k + 1` on false). For MAXIMIZE, make the condition "first infeasible" and subtract 1 — same loop, no ceiling mid.

Part 1 — Binary Search on Array Index

Detail Table

#	Name	Description	Interval	On <code>arr[k]==target</code>	Shrink left	Shrink right	Return
704	Binary Search	Find index of target; -1 if absent	Half-open	—	<code>i = k+1</code>	<code>j = k</code>	lower, then <code>arr[i]==target ? i : -1</code>
34	First & Last Position	First and last index of target	Half-open	—	<code>i = k+1</code>	<code>j = k</code>	<code>{lower, upper-1}</code>
35	Search Insert Position	Index where target is or would be inserted	Half-open	—	<code>i = k+1</code>	<code>j = k</code>	<code>i</code>
33	Search in Rotated Array	Find target after unknown-pivot rotation	Half-open	—	<code>i = k+1</code>	<code>j = k</code>	find pivot, then lowerBound; <code>nums[i]==target ? i : -1</code>
162	Find Peak Element	Any local peak index	Half-open*	—	<code>i = k+1</code>	<code>j = k</code>	<code>i</code>

#704 Binary Search

Description: Given a sorted array and a target, return its index. Return -1 if not found.

Template: `[i, j)` — run the loop, then **the answer is `i`**; just check the `i == 0` / `i == nums.length` edges before reading the array.

Intuition: Both bounds converge to a single index `i`. With `lowerBound`, `i` is the first slot `>= target`, so check `nums[i]`. With `upperBound`, `i` is one past the last `<= target`, so check `nums[i-1]`. Same loop, mirror edge.

Solution 1 — lowerBound (look at `i`):

Variables: `[i, j)` = half-open range over indices · `k` = midpoint · `condition(k) = nums[k] >= target` (first slot at-or-past target) · `target` = value sought **Pseudocode:**

```
i = 0, j = n
while i < j:
    k = midpoint
    if nums[k] < target: condition false, go right (i = k+1)
    else: condition true, k might be it (j = k)
# i = first index with nums[i] >= target
if i == n or nums[i] != target: return -1
return i
```

```

class Solution {
    public int search(int[] nums, int target) {
        int i = 0, j = nums.length;    // [i, j)
        // condition(k): nums[k] >= target
        while (i < j) {
            int k = i + (j - i) / 2;
            if (nums[k] < target) {      // condition false → go right
                i = k + 1;
            } else {                    // condition true → k might be the answer
                j = k;
            }
        }
        // i in [0, n]; guard i == n before reading nums[i]
        return (i == nums.length || nums[i] != target) ? -1 : i;
    }
}

```

Solution 2 — upperBound (look at `i - 1`):

Variables: `[i, j)` = half-open range over indices · `k` = midpoint · `condition(k)` = `nums[k] > target` (first slot strictly past target) · answer candidate is `i - 1` **Pseudocode:**

```

i = 0, j = n
while i < j:
    k = midpoint
    if nums[k] <= target: condition false, go right (i = k+1)
    else: condition true, k might be it (j = k)
# i = first index with nums[i] > target, so target (if present) is at i-1
if i == 0 or nums[i-1] != target: return -1
return i - 1

```

```

class Solution {
    public int search(int[] nums, int target) {
        int i = 0, j = nums.length;    // [i, j)
        // condition(k): nums[k] > target
        while (i < j) {
            int k = i + (j - i) / 2;
            if (nums[k] <= target) {      // condition false → go right
                i = k + 1;
            } else {                    // condition true → k might be the answer
                j = k;
            }
        }
        // i in [0, n]; the candidate is the element just before i — guard i == 0
        return (i == 0 || nums[i - 1] != target) ? -1 : i - 1;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

The `[i, j)` rule — always answer with `i`, then guard the edge:

- `i` always lands in `[0, nums.length]`.
- **lowerBound** answer is `nums[i]` → guard `i == nums.length` (target past the end) before reading; then confirm `nums[i] == target`.
- **upperBound** answer is `nums[i - 1]` → guard `i == 0` (target before the start) before reading; then confirm `nums[i - 1] == target`.
- The `nums[..] != target` check is still required because the bound only gives the *insert position*, not proof the value is present (e.g. `[1,3,5]`, target `2` → lowerBound `i = 1`, `nums[1] = 3 ≠ 2`).

#34 Find First and Last Position of Element in Sorted Array

Description: Return `[first, last]` index of target. Return `{-1, -1}` if absent.

Template: Half-open `[i, j)`. `lowerBound` vs `upperBound` differ by one char: `<` vs `<=`.

Intuition: Find the boundary where "`< target`" flips to "`≥ target`" (first position), and the next boundary just past target.

Variables: `[i, j)` = half-open range in each helper · `k` = midpoint · `lo` = `lowerBound` result (first `>= target`) ·

`lowerBound` `condition(k) = nums[k] >= target`; `upperBound` `condition(k) = nums[k] > target` **Pseudocode:**

```
lo = lowerBound(target)           # first index >= target
if lo == n or nums[lo] != target: return {-1, -1} # absent
return {lo, upperBound(target) - 1} # last = (first > target) - 1
lowerBound: [i, j) loop, go right while nums[k] < target, return i
upperBound: same loop, go right while nums[k] <= target, return i
```

```
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int lo = lowerBound(nums, target);           // first index >= target
        if (lo == nums.length || nums[lo] != target) { // ← edge guard, see below
            return new int[]{-1, -1};
        }
        return new int[]{lo, upperBound(nums, target) - 1}; // last = (first > target) - 1
    }
    private int lowerBound(int[] nums, int target) { // the one template, [i, j)
        int i = 0, j = nums.length;
        // condition(k): nums[k] >= target
        while (i < j) {
            int k = i + (j - i) / 2;
            if (nums[k] < target) { // condition false → go right
                i = k + 1;
            } else {                // condition true → k might be the answer
                j = k;
            }
        }
        return i;
    }
    private int upperBound(int[] nums, int target) { // same loop, "<" → "<="
        int i = 0, j = nums.length;
        // condition(k): nums[k] > target
        while (i < j) {
            int k = i + (j - i) / 2;
            if (nums[k] <= target) { // condition false → go right
                i = k + 1;
            } else {                // condition true → k might be the answer
                j = k;
            }
        }
        return i;
    }
}
```

Edge cases of `lo` (the `lowerBound` result), and why the one guard covers them all:

- `lo == nums.length` → target is bigger than everything ⇒ absent. The guard's first clause stops the OOB read.
- `lo == 0` but `nums[0] != target` → target is smaller than everything ⇒ absent. Caught by `nums[lo] != target`.
- `0 < lo < n` but `nums[lo] != target` → target falls in a gap (e.g. `[1,3,5]`, target `2`) ⇒ absent. Also caught by `nums[lo] != target`.
- otherwise `nums[lo] == target` → found; `upperBound - 1` is the last occurrence (guaranteed `> lo - 1`, so no separate check needed).

So a single `lo == n || nums[lo] != target` test handles all three "absent" buckets — you never need to special-case `lo == 0` separately. **Time** $O(\log n)$ | **Space** $O(1)$

#35 Search Insert Position

Description: Return the index where target is or would be inserted to keep the array sorted.

Template: Half-open `[i, j)` — identical to `lowerBound` from #34.

Intuition: The insert position is the first index whose value is $\geq$ target — exactly `lowerBound`.

Variables: `[i, j)` = half-open range over indices · `k` = midpoint · `condition(k) = arr[k] >= target` (first slot at-or-past target) · answer = `i` **Pseudocode:**

```
i = 0, j = n
while i < j:
    k = midpoint
    if arr[k] < target: condition false, go right (i = k+1)
    else: condition true, k might be it (j = k)
return i          # the insert position (lowerBound)
```

```
class Solution {
    public int searchInsert(int[] arr, int target) {
        int i = 0, j = arr.length;    // [i, j)
        // condition(k): arr[k] >= target
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < target) {      // condition false → go right
                i = k + 1;
            } else {                   // condition true → k might be the answer
                j = k;
            }
        }
        return i;                      // i in [0, n] = insert position
    }
}
```

Time $O(\log n)$ | **Space** $O(1)$

#33 Search in Rotated Sorted Array

Description: Array was sorted then rotated at an unknown pivot. Find target index, or -1.

Template: Two `[i, j)` searches, each converging to `i`: (1) find the pivot (min index), (2) `lowerBound` on the correct segment.

Intuition: First locate the rotation point with a boundary search, then the array splits into one sorted segment that must contain target — run a plain `lowerBound` there.

Variables: `[i, j)` = half-open range (reused for both searches) · `k` = midpoint · `pivot` = index of the minimum (rotation point); search-1 `condition(k) = nums[k] <= nums[n-1]` (`k` in the low run) · `lo / hi` = bounds of the sorted segment holding target; search-2 `condition(k) = nums[k] >= target` **Pseudocode:**

```

# 1) find pivot (index of minimum) over [0, n-1)
while i < j:
    k = midpoint
    if nums[k] > nums[n-1]: still in high run, go right (i = k+1)
    else: k might be the pivot (j = k)
pivot = i
# 2) pick the sorted segment that can contain target
if pivot==0 or target <= nums[n-1]: segment = [pivot, n) # right part
else: segment = [0, pivot) # left part
# lowerBound target within that segment
while i < j:
    k = midpoint
    if nums[k] < target: go right (i = k+1)
    else: j = k
return (nums[i] == target) ? i : -1

```

```

class Solution {
    public int search(int[] nums, int target) {
        int n = nums.length;
        // 1) pivot = index of the minimum, via [i, j)
        int i = 0, j = n - 1;
        // condition(k): nums[k] <= nums[n-1] (k is in the low run that holds the minimum)
        while (i < j) {
            int k = i + (j - i) / 2;
            if (nums[k] > nums[n - 1]) { // condition false → go right (still in high run)
                i = k + 1;
            } else { // condition true → k might be the pivot
                j = k;
            }
        }
        int pivot = i; // smallest element's index

        // 2) choose the sorted segment that can hold target, then lowerBound on it
        int lo, hi;
        if (pivot == 0 || target <= nums[n - 1]) { lo = pivot; hi = n; } // right segment
        else { lo = 0; hi = pivot; } // left segment
        i = lo; j = hi;
        // condition(k): nums[k] >= target
        while (i < j) {
            int k = i + (j - i) / 2;
            if (nums[k] < target) { // condition false → go right
                i = k + 1;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        return (i < n && nums[i] == target) ? i : -1; // ← [i, j) answer is i; verify hit
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

Both loops are the standard `[i, j)` template; the only post-step is the same `i < n && nums[i] == target` verification used by #704.

#162 Find Peak Element

Description: Return any index where $\text{arr}[k] > \text{arr}[k-1]$ and $\text{arr}[k] > \text{arr}[k+1]$. Edges are $-\infty$.

Template: Half-open $[i, j)$ with $j = n-1$. Binary search on slope: upward $\rightarrow$ peak is right, downward $\rightarrow$ peak is here or left.

Intuition: Always walk uphill — an upward slope guarantees a peak to the right, so the slope direction is the "condition".

Variables: $[i, j)$ = half-open range with $j = n-1$ (loop reads $\text{arr}[k+1]$) · k = midpoint · $\text{condition}(k) = \text{arr}[k] \geq \text{arr}[k+1]$ (slope down $\rightarrow$ peak at k or left) **Pseudocode:**

```
i = 0, j = n-1          # j = n-1 because we read arr[k+1]
while i < j:
    k = midpoint
    if arr[k] < arr[k+1]: slope up, condition false, go right (i = k+1)
    else: slope down, k might be a peak (j = k)
return i                # a peak index
```

```
class Solution {
    public int findPeakElement(int[] arr) {
        int i = 0, j = arr.length - 1;    // [i, j); reads arr[k+1] so j = n-1
        // condition(k): arr[k] >= arr[k+1] (slope down  $\rightarrow$  peak at k or to the left)
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < arr[k + 1]) {    // condition false  $\rightarrow$  go right (slope up)
                i = k + 1;
            } else {                    // condition true  $\rightarrow$  k might be the answer
                j = k;
            }
        }
        return i;
    }
}
```

Time $O(\log n)$ | **Space** $O(1)$

Part 2 — Binary Search on Answer Space

Search on the **answer value** itself, not an array index. All use half-open $[i, j)$ with $\text{while} (i < j)$.

Detail Table

#	Name	Description	Variant	i init	j init	Mid	Condition true	Condition false	Helper checks
875	Koko Eating Bananas	Min speed to finish all piles in h hours	MINIMIZE	1	max(piles)	floor	j = k	i = k+1	hours <= h
1011	Ship Within D Days	Min capacity to ship all packages in d days	MINIMIZE	max(w)	sum(w)	floor	j = k	i = k+1	days <= d
410	Split Array Largest Sum	Min largest sum splitting array into m parts	MINIMIZE	max(nums)	sum(nums)	floor	j = k	i = k+1	parts <= m
1283	Smallest Divisor	Smallest divisor so $\text{ceil-div-sum} \leq \text{threshold}$	MINIMIZE	1	max(nums)	floor	j = k	i = k+1	divSum <= threshold
1231	Divide Chocolate	Max min-sweetness cutting into k+1 pieces	MAXIMIZE	min(s)	sum(s)/(k+1)+1	floor	j = k	i = k+1	pieces < k+1 (first infeasible), return i-1

875 / 1011 / 410 / 1283 → identical structure, differ only in search bounds and condition helper.

1231 → MAXIMIZE: same floor-mid [i, j) loop, but condition(k) = !feasible(k) (first infeasible) and return i - 1.

#875 Koko Eating Bananas

Description: Minimum eating speed so Koko finishes all piles in h hours (one pile per hour, at most speed bananas eaten).

Search space: [1, max(piles)]

Intuition: Feasibility is monotonic in speed (faster always works), so binary search the speed axis for the smallest that finishes in time.

Variables: [i, j) = half-open range over candidate speeds · k = midpoint speed · condition(k) = canFinish(speed=k) (hours <= h); MINIMIZE → answer = i

Pseudocode:

```

i = 1, j = max(piles)      # search the speed axis
while i < j:
    k = midpoint
    if not canFinish(k): too slow, condition false, go right (i = k+1)
    else: k might be the answer (j = k)
return i                    # smallest feasible speed
# canFinish(speed): sum of ceil(pile/speed) hours, true if <= h

```

```

class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int i = 1, j = Arrays.stream(piles).max().getAsInt(); // [i, j) over speeds
        // condition(k): canFinish(speed=k) within h hours
        while (i < j) {
            int k = i + (j - i) / 2;
            if (!canFinish(piles, k, h)) { // condition false → go right (too slow)
                i = k + 1;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        return i; // smallest feasible speed
    }
    private boolean canFinish(int[] piles, int speed, int h) {
        int hours = 0;
        for (int p : piles) hours += (p + speed - 1) / speed;
        return hours <= h;
    }
}

```

Time $O(n \log(\max(\text{piles})))$ | **Space** $O(1)$

#1011 Capacity to Ship Packages Within D Days

Description: Minimum ship capacity to deliver all packages (in order) within d days.

Search space: $[\max(\text{weights}), \text{sum}(\text{weights})]$ — must fit heaviest; worst case ships one per day.

Intuition: Bigger capacity is always feasible, so binary search capacity for the smallest that ships within the day budget.

Variables: $[i, j)$ = half-open range over capacities · k = midpoint capacity · $\text{condition}(k) = \text{canShip}(\text{capacity}=k)$
(days needed $\leq$ days); MINIMIZE → answer = i **Pseudocode:**

```

i = max(weights), j = sum(weights)      # smallest fits heaviest; largest ships all in one day
while i < j:
    k = midpoint
    if not canShip(k): too small, condition false, go right (i = k+1)
    else: k might be the answer (j = k)
return i      # smallest feasible capacity
# canShip(cap): greedily fill days, count days, true if <= days

```

```

class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int i = Arrays.stream(weights).max().getAsInt();
        int j = Arrays.stream(weights).sum(); // [i, j) over capacities
        // condition(k): canShip(capacity=k) within days
        while (i < j) {
            int k = i + (j - i) / 2;
            if (!canShip(weights, k, days)) { // condition false → go right (too small)
                i = k + 1;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        return i; // smallest feasible capacity
    }
    private boolean canShip(int[] weights, int cap, int days) {
        int d = 1, load = 0;
        for (int w : weights) {
            if (load + w > cap) { d++; load = 0; }
            load += w;
        }
        return d <= days;
    }
}

```

Time $O(n \log(\text{sum}(\text{weights})))$ | **Space** $O(1)$

#410 Split Array Largest Sum

Description: Split array into m non-empty subarrays to minimize the largest subarray sum.

Search space: $[\max(\text{nums}), \text{sum}(\text{nums})]$ — identical bounds to #1011.

Intuition: A larger allowed max-sum needs fewer parts, so binary search that cap for the smallest splittable into $\leq m$ parts.

Variables: $[i, j)$ = half-open range over max-sum caps · k = midpoint cap · $\text{condition}(k)$ = `canSplit` into $\leq m$ parts with cap k ; MINIMIZE → answer = i **Pseudocode:**

```

i = max(nums), j = sum(nums)    # cap must hold biggest element; whole array is one part at most
while i < j:
    k = midpoint
    if not canSplit(k): cap too small, condition false, go right (i = k+1)
    else: k might be the answer (j = k)
return i    # smallest feasible cap
# canSplit(cap): greedily form parts not exceeding cap, true if parts <= m

```

```

class Solution {
    public int splitArray(int[] nums, int m) {
        int i = Arrays.stream(nums).max().getAsInt();
        int j = Arrays.stream(nums).sum();    // [i, j) over max-sum caps
        // condition(k): canSplit into <= m parts with cap=k
        while (i < j) {
            int k = i + (j - i) / 2;
            if (!canSplit(nums, k, m)) {    // condition false → go right (cap too small)
                i = k + 1;
            } else {                        // condition true → k might be the answer
                j = k;
            }
        }
        return i;                            // smallest feasible cap
    }
    private boolean canSplit(int[] nums, int cap, int m) {
        int parts = 1, sum = 0;
        for (int n : nums) {
            if (sum + n > cap) { parts++; sum = 0; }
            sum += n;
        }
        return parts <= m;
    }
}

```

Time $O(n \log(\text{sum}(\text{nums})))$ | **Space** $O(1)$

#1283 Find the Smallest Divisor Given a Threshold

Description: Smallest positive divisor such that `sum of ceil(nums[i] / divisor) <= threshold`.

Search space: `[1, max(nums)]`

Intuition: A bigger divisor shrinks the sum, so the condition flips false→true; binary search for the smallest divisor that fits the threshold.

Variables: `[i, j)` = half-open range over divisors · `k` = midpoint divisor · `condition(k) = divSum(k) <= threshold`;
MINIMIZE → answer = `i` **Pseudocode:**

```

i = 1, j = max(nums)    # bigger divisor shrinks the ceil-sum
while i < j:
    k = midpoint
    if divSum(k) > threshold: divisor too small, condition false, go right (i = k+1)
    else: k might be the answer (j = k)
return i                # smallest feasible divisor
# divSum(d): sum of ceil(num/d) over all nums

```

```

class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int i = 1, j = Arrays.stream(nums).max().getAsInt(); // [i, j) over divisors
        // condition(k): divSum(divisor=k) <= threshold
        while (i < j) {
            int k = i + (j - i) / 2;
            if (divSum(nums, k) > threshold) { // condition false → go right (divisor too small)
                i = k + 1;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        return i; // smallest feasible divisor
    }
    private int divSum(int[] nums, int d) {
        int sum = 0;
        for (int n : nums) sum += (n + d - 1) / d;
        return sum;
    }
}

```

Time $O(n \log(\max(\text{nums})))$ | **Space** $O(1)$

#1231 Divide Chocolate ← MAXIMIZE (the one that differs)

Description: Cut chocolate into $k+1$ pieces; keep the minimum-sweetness piece. Maximize that minimum.

Search space: $[\min(s), \text{sum}(s)/(k+1)]$

Intuition: A larger target-minimum yields fewer pieces, so feasibility flips true→false. MAXIMIZE = search the FIRST INFEASIBLE minimum and step back one: $\text{condition}(k) = \text{!canDivide}(k)$ (pieces $< k+1$), false...false TRUE...true. Half-open $[\text{lo}, \text{hi}+1)$, return $i - 1$ — no ceiling mid needed.

Variables: $[i, j)$ = half-open range over candidate minimum-sweetness values · m = midpoint candidate · $\text{condition}(m) = \text{!canDivide}$ into $k+1$ pieces each $\geq m$ (first INFEASIBLE); MAXIMIZE → answer = $i - 1$

Pseudocode:

```

i = min(s), j = sum(s)/(k+1) + 1      # half-open [lo, hi+1)
while i < j:
    m = midpoint
    if canDivide(m): feasible, condition false, go right (i = m+1)
    else: infeasible, m might be the boundary (j = m)
return i - 1      # last feasible minimum
# canDivide(minSweet): greedily count pieces each summing >= minSweet, true if count >= k+1

```



```

class Solution {
    public int maximizeSweetness(int[] s, int k) {
        int i = Arrays.stream(s).min().getAsInt();
        int j = Arrays.stream(s).sum() / (k + 1) + 1; // [i, j) over candidate minimums
        // condition(m): !canDivide into k+1 pieces each summing >= m (first INFEASIBLE minimum)
        while (i < j) {
            int m = i + (j - i) / 2;
            if (canDivide(s, k + 1, m)) { // feasible → condition false → go right
                i = m + 1;
            } else { // infeasible → condition true → m might be the boundary
                j = m;
            }
        }
        return i - 1; // last feasible minimum (i >= lo, so i-1 valid)
    }
    private boolean canDivide(int[] s, int pieces, int minSweet) {
        int count = 0, sum = 0;
        for (int x : s) {
            sum += x;
            if (sum >= minSweet) { count++; sum = 0; }
        }
        return count >= pieces;
    }
}

```

Time $O(n \log(\text{sum}(s)))$ | **Space** $O(1)$

MINIMIZE vs MAXIMIZE Side by Side

	MINIMIZE (875,1011,410,1283)	MAXIMIZE (1231)
Mid:	<code>floor i + (j-i) / 2</code>	<code>floor i + (j-i) / 2</code>
condition(k):	<code>feasible(k)</code>	<code>!feasible(k)</code> (first infeasible)
If condition false:	<code>i = k + 1</code>	<code>i = k + 1</code>
If condition true:	<code>j = k</code>	<code>j = k</code>
Range:	<code>[lo, hi] → [lo, hi+1)</code>	<code>[lo, hi] → [lo, hi+1)</code>
Return:	<code>i</code>	<code>i - 1</code>

Both are the SAME `[i, j)` floor-mid loop. MAXIMIZE just defines `condition(k) = !feasible(k)` (the first infeasible value) over `[lo, hi+1)` and returns `i - 1`. No ceiling mid is ever needed, so the `i+1 == j` infinite-loop trap of `i = k` simply never arises.

Additional Reference Problems

#4 Median of Two Sorted Arrays

Description: Find the median of two sorted arrays of sizes m and n in $O(\log(m+n))$ time.

Template: Half-open `[i, j)` — lowerBound for the partition cut in the smaller array (first cut where `left2 <= right1`).

Intuition: Pick a split of the smaller array; the rest of the split is forced. Shrink the partition until the left half's $\text{max} \leq$ the right half's min , then the median falls out of the boundary values.

Variables: `[i, j)` = half-open range over cuts k in the smaller array · k = cut in `nums1` · `cut2 = half - k` = forced cut in `nums2` · `left1 / right1 / left2 / right2` = boundary values around the two cuts · `condition(k) = left2 <= right1` (cut k is a valid partition) **Pseudocode:**

```

ensure nums1 is the smaller array (else recurse swapped)
half = (m+n+1)/2
i = max(0, half-n), j = min(m, half) + 1      # cuts keeping cut2 in [0, n]
while i < j:
    k = midpoint; cut2 = half - k
    right1 = nums1[k] (or +inf), left2 = nums2[cut2-1] (or -inf)
    if left2 > right1: condition false, go right (i = k+1)
    else: valid partition, k might be it (j = k)
# i is the valid cut; read left1/right1/left2/right2 around both cuts
if total length odd: return max(left1, left2)
else: return (max(left1, left2) + min(right1, right2)) / 2

```

```

class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) { // ← VARIATION: search smaller array
            return findMedianSortedArrays(nums2, nums1);
        }
        int m = nums1.length, n = nums2.length;
        int half = (m + n + 1) / 2;
        // [i, j): find the first cut k in nums1 with condition(k) = (left2 <= right1).
        // As k grows, right1 = nums1[k] grows and left2 = nums2[half-k-1] shrinks, so
        // condition is false...false TRUE...true; the first true cut is the correct partition,
        // where simultaneously left1 <= right2 holds.
        // k must keep cut2 = half-k in [0, n], so k ranges over [max(0, half-n), min(m, half)].
        int i = Math.max(0, half - n), j = Math.min(m, half) + 1; // [i, j) over cuts
        // condition(k): left2 <= right1 (cut k is a valid partition)
        while (i < j) {
            int k = i + (j - i) / 2; // cut in nums1
            int cut2 = half - k; // cut in nums2
            int right1 = (k == m) ? Integer.MAX_VALUE : nums1[k];
            int left2 = (cut2 == 0) ? Integer.MIN_VALUE : nums2[cut2 - 1];
            if (left2 > right1) { // condition false → go right (too few from nums1)
                i = k + 1;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        // i is the smallest cut with left2 <= right1 (the valid partition)
        int k = i, cut2 = half - i;
        int left1 = (k == 0) ? Integer.MIN_VALUE : nums1[k - 1];
        int right1 = (k == m) ? Integer.MAX_VALUE : nums1[k];
        int left2 = (cut2 == 0) ? Integer.MIN_VALUE : nums2[cut2 - 1];
        int right2 = (cut2 == n) ? Integer.MAX_VALUE : nums2[cut2];
        if (((m + n) & 1) == 1) {
            return Math.max(left1, left2);
        }
        return (Math.max(left1, left2) + Math.min(right1, right2)) / 2.0;
    }
}

```

Time $O(\log(\min(m, n)))$ | **Space** $O(1)$

#69 Sqrt(x)

Description: Compute the integer square root of a non-negative integer x without using `sqrt()`.

Template: Answer space MAXIMIZE — largest k with $k*k \leq x$.

Intuition: Feasibility $k*k \leq x$ is true...true then false as k grows. MAXIMIZE = find the FIRST INFEASIBLE k with $\text{condition}(k) = k*k > x$ (false...false TRUE...true), over $[1, x+1)$, and return $i - 1$ — the largest k still feasible. No

ceiling mid needed.

Variables: `[i, j)` = half-open range over candidate roots · `k` = midpoint candidate · `condition(k) = k*k > x` (first INFEASIBLE); MAXIMIZE → answer = `i - 1` **Pseudocode:**

```
if x < 2: return x
i = 1, j = x + 1      # half-open [1, x+1)
while i < j:
    k = midpoint
    if k*k <= x: feasible, condition false, go right (i = k+1)
    else: infeasible, k might be the boundary (j = k)
return i - 1          # largest k with k*k <= x
```

```
class Solution {
    public int mySqrt(int x) {
        if (x < 2) {
            return x;
        }
        int i = 1, j = x + 1;           // [i, j); condition(k): k*k > x
        while (i < j) {
            int k = i + (j - i) / 2;
            if ((long) k * k <= x) {    // feasible → condition false → go right
                i = k + 1;
            } else {                   // infeasible → condition true → k might be the boundary
                j = k;
            }
        }
        return i - 1;                  // largest k with k*k <= x (i >= 1, so i-1 valid)
    }
}
```

Time $O(\log x)$ | **Space** $O(1)$

#74 Search a 2D Matrix

Description: Search for a target in an $m \times n$ matrix where each row is sorted and each row's first element > previous row's last.

Template: Half-open `[i, j)` — treat the matrix as one flat sorted array of length $m*n$ and run `lowerBound`.

Intuition: The layout means the whole matrix reads as a single sorted sequence; map a flat index to `(row, col)` and run a standard `[i, j)` `lowerBound`, then verify the landed value.

Variables: `[i, j)` = half-open range over flat indices $0..m*n$ · `k` = midpoint flat index, mapped to `matrix[k/n][k%n]` · `condition(k) = value >= target` · `target = value sought` **Pseudocode:**

```
i = 0, j = m*n      # treat matrix as one flat sorted array
while i < j:
    k = midpoint
    value = matrix[k/n][k%n]      # flatten index to 2D
    if value < target: condition false, go right (i = k+1)
    else: k might be it (j = k)
# i is lowerBound; verify the landed cell
return i < m*n and matrix[i/n][i%n] == target
```

```

class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length;
        int i = 0, j = m * n; // [i, j) over flat indices
        // condition(k): matrix[k/n][k%n] >= target
        while (i < j) {
            int k = i + (j - i) / 2;
            int value = matrix[k / n][k % n]; // ← VARIATION: flatten index to 2D
            if (value < target) { // condition false → go right
                i = k + 1;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        // i in [0, m*n]; guard i == m*n, then verify the landed cell equals target
        return i < m * n && matrix[i / n][i % n] == target;
    }
}

```

Time $O(\log(m \cdot n))$ | **Space** $O(1)$

#81 Search in Rotated Sorted Array II

Description: Search in a sorted, rotated array that may contain duplicates.

Template: Half-open $[i, j)$ — like #33, but duplicates force a linear shrink when $\text{arr}[i] == \text{arr}[k] == \text{arr}[j-1]$. The last in-range element is $\text{arr}[j-1]$.

Intuition: Duplicates can make both ends equal to the midpoint so neither half looks sorted; in that ambiguous case shrink both bounds by one and retry.

Variables: $[i, j)$ = half-open range (last in-range element is $\text{arr}[j-1]$) · k = midpoint · branching search (no single monotone condition because duplicates break monotonicity) **Pseudocode:**

```

i = 0, j = n      # last element is arr[j-1]
while i < j:
    k = midpoint
    if arr[k] == target: return true
    if arr[i] == arr[k] == arr[j-1]: ambiguous duplicates, shrink both ends (i++, j--)
    elif left half sorted (arr[i] <= arr[k]):
        if target in [arr[i], arr[k]): go left (j = k) else go right (i = k+1)
    else: # right half sorted
        if target in (arr[k], arr[j-1]]: go right (i = k+1) else go left (j = k)
return false

```

```

class Solution {
    public boolean search(int[] arr, int target) {
        int i = 0, j = arr.length;           // [i, j); last element is arr[j-1]
        // No single monotone condition(k) here: duplicates break monotonicity, so this is a
        // branching rotated search (not the canonical first-true template). Bounds stay [i, j).
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] == target) {
                return true;
            }
            if (arr[i] == arr[k] && arr[k] == arr[j - 1]) {    // ← VARIATION: ambiguous duplicates
                i++;
                j--;
            } else if (arr[i] <= arr[k]) {                    // left half sorted
                if (arr[i] <= target && target < arr[k]) {
                    j = k;
                } else {
                    i = k + 1;
                }
            } else {                                          // right half sorted
                if (arr[k] < target && target <= arr[j - 1]) {
                    i = k + 1;
                } else {
                    j = k;
                }
            }
        }
        return false;
    }
}

```

Time $O(\log n)$ average, $O(n)$ worst | **Space** $O(1)$

#153 Find Minimum in Rotated Sorted Array

Description: Find the minimum element in a sorted, rotated array with no duplicates.

Template: Half-open $[i, j]$ on slope — compare midpoint to the right end to pick the unsorted half.

Intuition: The minimum is the single point where the order breaks; if $arr[k] > arr[j]$ the break is to the right, otherwise the min is at k or left.

Variables: $[i, j]$ = closed range (compares to the running right end $arr[j]$) · k = midpoint · $condition(k) = arr[k] \leq arr[j]$ (min is at k or to its left) **Pseudocode:**

```

i = 0, j = n-1      # compare midpoint to the right end
while i < j:
    k = midpoint
    if arr[k] > arr[j]: break is to the right, condition false, go right (i = k+1)
    else: min at k or left, k might be it (j = k)
return arr[i]       # the minimum

```

```

class Solution {
    public int findMin(int[] arr) {
        int i = 0, j = arr.length - 1; // [i, j]; compares to running right end
        // condition(k): arr[k] <= arr[j] (min is at k or to its left)
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] > arr[j]) { // condition false → go right (break is to the right)
                i = k + 1;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        return arr[i];
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#240 Search a 2D Matrix II

Description: Search for a target in an $m \times n$ matrix where rows and columns are both sorted.

Template: Staircase search from the top-right corner (not a halving binary search, but the canonical $O(m+n)$ solution).

Intuition: From the top-right, moving left strictly decreases and moving down strictly increases, so each comparison eliminates a full row or column.

Variables: `row` = current row (starts at 0) · `col` = current column (starts at last) · staircase walk from the top-right corner · `target` = value sought **Pseudocode:**

```

start at top-right corner (row=0, col=last)
while inside the matrix:
    value = matrix[row][col]
    if value == target: return true
    elif value > target: move left (col--) # current column too big
    else: move down (row++) # current row too small
return false

```

```

class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int row = 0, col = matrix[0].length - 1; // ← VARIATION: start top-right corner
        while (row < matrix.length && col >= 0) {
            int value = matrix[row][col];
            if (value == target) {
                return true;
            } else if (value > target) {
                col--;
            } else {
                row++;
            }
        }
        return false;
    }
}

```

Time $O(m + n)$ | **Space** $O(1)$

#278 First Bad Version

Description: Find the first bad version using a provided `isBadVersion(n)` API. Minimize API calls.

Template: Half-open `[i, j)` — find the first index where `isBadVersion` flips false→true.

Intuition: Versions are good...good then bad...bad once corrupted; this is the classic first-true boundary search.

Variables: `[i, j]` = closed range over versions `1..n` · `k` = midpoint version · `condition(k)` = `isBadVersion(k)` (good...good then bad...bad) **Pseudocode:**

```
i = 1, j = n
while i < j:
    k = midpoint
    if not isBadVersion(k): still good, condition false, go right (i = k+1)
    else: k might be the first bad (j = k)
return i      # first bad version
```

```
/* The isBadVersion API is defined in the parent class VersionControl.
boolean isBadVersion(int version); */
public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int i = 1, j = n;           // [i, j]
        // condition(k): isBadVersion(k)
        while (i < j) {
            int k = i + (j - i) / 2;
            if (!isBadVersion(k)) { // condition false → go right (still good)
                i = k + 1;
            } else {                // condition true → k might be the answer
                j = k;
            }
        }
        return i;                   // first bad version
    }
}
```

Time $O(\log n)$ | **Space** $O(1)$

#374 Guess Number Higher or Lower

Description: Guess the number between 1 and `n` using a provided `guess()` API. Minimize calls.

Template: Half-open `[i, j)` — find the first `k` with `guess(k) <= 0`, which is exactly the pick.

Intuition: `guess(k)` returns `-1` when the pick is lower than `k`, `1` when higher, `0` on a hit. So `guess(k) <= 0` is false... false TRUE...true as `k` grows, and the first true `k` is the answer.

Variables: `[i, j)` = half-open range over candidates `1..n` · `k` = midpoint guess · `condition(k)` = `guess(k) <= 0` (`k` is the pick or pick is lower) **Pseudocode:**

```
i = 1, j = n + 1
while i < j:
    k = midpoint
    if guess(k) > 0: pick is higher, condition false, go right (i = k+1)
    else: k is the pick or pick is lower (j = k)
return i      # the picked number
```

```

/* The guess API is defined in the parent class GuessGame.
   int guess(int num); returns -1 (pick is lower), 1 (pick is higher), or 0 (hit). */
public class Solution extends GuessGame {
    public int guessNumber(int n) {
        int i = 1, j = n + 1; // [i, j) over candidates 1..n
        // condition(k): guess(k) <= 0 (k is the pick, or pick is lower)
        while (i < j) {
            int k = i + (j - i) / 2;
            if (guess(k) > 0) { // condition false → go right (pick is higher)
                i = k + 1;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        return i; // first k with guess(k) <= 0 = the pick
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#475 Heaters

Description: Given house and heater positions, find the minimum heater radius so every house is covered by some heater.

Template: Half-open $[i, j)$ — for each house, lowerBound the nearest heater $\geq$ house, then compare it with the left neighbor; the answer is the max over all houses of the closest distance.

Intuition: Each house needs the closest heater; the required radius is the largest of those closest distances. Find each house's nearest heater by binary search.

Variables: $[i, j)$ = half-open range over heater indices (per house) · k = midpoint · $\text{condition}(k) = \text{heaters}[k] \geq \text{house}$ (lowerBound) · distance = closest heater distance for this house · result = max over all houses **Pseudocode:**

```

sort heaters; result = 0
for each house:
    lowerBound first heater >= house over [0, n):
        if heaters[k] < house: go right (i = k+1) else j = k
    distance = +inf
    if i < n: distance = heaters[i] - house # heater to the right
    if i > 0: distance = min(distance, house - heaters[i-1]) # also check left neighbor
    result = max(result, distance)
return result # the required radius

```



```

class Solution {
    public int findRadius(int[] houses, int[] heaters) {
        Arrays.sort(heaters);
        int n = heaters.length;
        int result = 0;
        for (int house : houses) {
            int i = 0, j = n; // [i, j): lowerBound first heater >= house
            // condition(k): heaters[k] >= house
            while (i < j) {
                int k = i + (j - i) / 2;
                if (heaters[k] < house) { // condition false → go right
                    i = k + 1;
                } else { // condition true → k might be the answer
                    j = k;
                }
            }
            int distance = Integer.MAX_VALUE;
            if (i < n) { // heater at or to the right of house
                distance = heaters[i] - house;
            }
            if (i > 0) { // ← VARIATION: also check left neighbor
                distance = Math.min(distance, house - heaters[i - 1]);
            }
            result = Math.max(result, distance);
        }
        return result;
    }
}

```

Time $O((m + n) \log m)$ | **Space** $O(1)$

#540 Single Element in a Sorted Array

Description: Find the single non-duplicate element in a sorted array where all others appear twice. $O(\log n)$.

Template: Half-open $[i, j]$ on even indices — a pair starting at an even index is intact iff it lies before the single element.

Intuition: Before the unique element each pair starts at an even index; after it, the parity shifts. Force the midpoint even and check whether its partner matches to pick a half.

Variables: $[i, j]$ = closed range on even indices · k = midpoint forced even · $\text{condition}(k) = \text{arr}[k] \neq \text{arr}[k+1]$
 (single is at k or to its left) **Pseudocode:**

```

i = 0, j = n-1
while i < j:
    k = midpoint; if k is odd, drop to k-1 (force even)
    if arr[k] == arr[k+1]: pair intact, single is to the right (i = k+2)
    else: pair broken, single at k or left (j = k)
return arr[i] # the single element

```

```

class Solution {
    public int singleNonDuplicate(int[] arr) {
        int i = 0, j = arr.length - 1; // [i, j] on even indices
        // condition(k): arr[k] != arr[k+1] with k forced even (the single is at k or to its left)
        while (i < j) {
            int k = i + (j - i) / 2;
            if ((k & 1) == 1) { // ← VARIATION: force k even
                k--;
            }
            if (arr[k] == arr[k + 1]) { // condition false → go right (pair intact; +2 stays even)
                i = k + 2;
            } else { // condition true → k might be the answer
                j = k;
            }
        }
        return arr[i];
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#658 Find K Closest Elements

Description: Find k integers in a sorted array closest to x , ordered by closeness then value.

Template: Half-open $[i, j]$ — binary search the left boundary of the size- k window.

Intuition: The answer is a contiguous window of size k ; search for its start by comparing the distances of the two window edges to x and sliding toward the closer side.

Variables: $[i, j]$ = closed range over window-start indices $0..n-k$ · mid = candidate window start · $condition(mid)$ = $x - arr[mid] \leq arr[mid+k] - x$ (left edge at mid is closer than the right edge) **Pseudocode:**

```

i = 0, j = n-k          # window start range
while i < j:
    mid = midpoint
    if x - arr[mid] > arr[mid+k] - x: right edge closer, shift window right (i = mid+1)
    else: left edge closer, mid might be the start (j = mid)
collect arr[i .. i+k-1] as the answer window

```

```

class Solution {
    public List<Integer> findClosestElements(int[] arr, int k, int x) {
        int i = 0, j = arr.length - k; // ← VARIATION: search window start [i, j]
        // condition(k=mid): x - arr[mid] <= arr[mid+k] - x (left edge of window at mid is closer)
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (x - arr[mid] > arr[mid + k] - x) { // condition false → go right (right edge closer)
                i = mid + 1;
            } else { // condition true → mid might be the answer
                j = mid;
            }
        }
        List<Integer> result = new ArrayList<>();
        for (int p = i; p < i + k; p++) {
            result.add(arr[p]);
        }
        return result;
    }
}

```

Time $O(\log(n - k) + k)$ | **Space** $O(k)$

#719 Find K-th Smallest Pair Distance

Description: Find the k-th smallest absolute distance among all pairs in the array.

Template: Answer space MINIMIZE — smallest distance d with at least k pairs $\leq d$.

Intuition: The count of pairs with distance $\leq d$ is monotonic in d, so binary search the distance axis; count pairs $\leq d$ with a sliding window over the sorted array.

Variables: `[i, j]` = closed range over candidate distances · `mid` = candidate distance · `condition(mid)` = `countPairs(dist  $\leq$  mid)  $\geq$  k`; MINIMIZE $\rightarrow$ answer = `i` **Pseudocode:**

```
sort nums
i = 0, j = max - min          # search the distance axis
while i < j:
    mid = midpoint
    if countPairs(mid) < k: too few pairs, condition false, go right (i = mid+1)
    else: mid might be the answer (j = mid)
return i                      # smallest distance with  $\geq$  k pairs
# countPairs(maxDist): sliding window over sorted nums, sum (right - left) per right
```

```
class Solution {
    public int smallestDistancePair(int[] nums, int k) {
        Arrays.sort(nums);
        int i = 0, j = nums[nums.length - 1] - nums[0];    // ← VARIATION: search distance value [i, j]
        // condition(k=mid): countPairs(dist  $\leq$  mid)  $\geq$  k
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (countPairs(nums, mid) < k) { // condition false  $\rightarrow$  go right (too few pairs)
                i = mid + 1;
            } else { // condition true  $\rightarrow$  mid might be the answer
                j = mid;
            }
        }
        return i; // smallest distance with  $\geq$  k pairs
    }
    private int countPairs(int[] nums, int maxDist) {
        int count = 0, left = 0;
        for (int right = 0; right < nums.length; right++) {
            while (nums[right] - nums[left] > maxDist) {
                left++;
            }
            count += right - left;
        }
        return count;
    }
}
```

Time $O(n \log n + n \log(\text{maxDist}))$ | **Space** $O(1)$

#852 Peak Index in a Mountain Array

Description: Find the peak index in a mountain array (element greater than both neighbors).

Template: Half-open `[i, j]` on slope — walk uphill toward the peak.

Intuition: An upward slope means the peak is strictly to the right; a downward slope means the peak is here or to the left.

Variables: `[i, j]` = closed range with `j = n-1` (loop reads `arr[k+1]`) · `k` = midpoint · `condition(k) = arr[k]  $\geq$  arr[k+1]` (slope down $\rightarrow$ peak at k or left) **Pseudocode:**

```

i = 0, j = n-1          # j = n-1 because we read arr[k+1]
while i < j:
    k = midpoint
    if arr[k] < arr[k+1]: slope up, condition false, go right (i = k+1)
    else: slope down, k might be the peak (j = k)
return i                # peak index

```

```

class Solution {
    public int peakIndexInMountainArray(int[] arr) {
        int i = 0, j = arr.length - 1;           // [i, j]; reads arr[k+1] so j = n-1
        // condition(k): arr[k] >= arr[k+1] (slope down → peak at k or to its left)
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < arr[k + 1]) {           // condition false → go right (slope up)
                i = k + 1;
            } else {                             // condition true → k might be the answer
                j = k;
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#1060 Missing Element in Sorted Array

Description: Find the k th missing number in a sorted array.

Template: Half-open $[i, j]$ — the count of missing numbers before index k is $\text{arr}[k] - \text{arr}[0] - k$, which is monotonic.

Intuition: Missing count at each index increases monotonically, so binary search for the first index whose missing count is $\geq k$, then offset from the previous element.

Variables: $[i, j]$ = closed range over indices · mid = midpoint · $\text{condition}(\text{mid}) = \text{missingCount}(\text{mid}) \geq k$ where $\text{missingCount}(\text{idx}) = \text{nums}[\text{idx}] - \text{nums}[0] - \text{idx}$ **Pseudocode:**

```

i = 0, j = n-1
while i < j:
    mid = midpoint
    if missingCount(mid) < k: too few missing, condition false, go right (i = mid+1)
    else: mid might be it (j = mid)
if even nums[n-1] has < k missing: answer lies beyond the array, offset from last element
else: offset the kth missing from nums[i-1] using its missing count

```

```

class Solution {
    public int missingElement(int[] nums, int k) {
        int n = nums.length;
        int i = 0, j = n - 1; // [i, j]
        // condition(k=mid): missingCount(mid) >= k
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (missingCount(nums, mid) < k) { // condition false → go right (too few missing)
                i = mid + 1;
            } else { // condition true → mid might be the answer
                j = mid;
            }
        }
        if (missingCount(nums, i) < k) { // ← VARIATION: kth missing beyond array end
            return nums[n - 1] + (k - missingCount(nums, n - 1));
        }
        return nums[i - 1] + (k - missingCount(nums, i - 1));
    }
    private int missingCount(int[] nums, int index) {
        return nums[index] - nums[0] - index;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#1428 Leftmost Column with at Least a One

Description: Find the leftmost column with at least one '1' in a binary matrix (rows sorted, accessed via BinaryMatrix API).

Template: Staircase walk from the top-right corner across rows that are individually sorted.

Intuition: Each row is sorted 0...0 1...1; starting top-right, move left on a 1 (column might be even further left) and down on a 0, tracking the leftmost 1 seen.

Variables: `row` = current row (starts at 0) · `col` = current column (starts at last) · `result` = leftmost column with a 1 seen so far (start -1) · staircase walk from the top-right corner **Pseudocode:**

```

start at top-right corner (row=0, col=last); result = -1
while inside the matrix:
    if cell is 1: record col as result; move left (col--) # maybe further left
    else: move down (row++) # no 1 in this row at/after col
return result

```

```

/* interface BinaryMatrix {
    public int get(int row, int col);
    public List<Integer> dimensions();
} */
class Solution {
    public int leftMostColumnWithOne(BinaryMatrix binaryMatrix) {
        List<Integer> dim = binaryMatrix.dimensions();
        int rows = dim.get(0), cols = dim.get(1);
        int row = 0, col = cols - 1; // ← VARIATION: start top-right corner
        int result = -1;
        while (row < rows && col >= 0) {
            if (binaryMatrix.get(row, col) == 1) {
                result = col;
                col--;
            } else {
                row++;
            }
        }
        return result;
    }
}

```

Time $O(\text{rows} + \text{cols})$ | **Space** $O(1)$

#1482 Minimum Number of Days to Make m Bouquets

Description: Find the minimum number of days to make m bouquets of k adjacent flowers.

Template: Answer space MINIMIZE — smallest day count d that yields $\geq m$ bouquets.

Intuition: Waiting more days only adds bloomed flowers, so feasibility is monotonic; binary search the day axis and greedily count adjacent runs of bloomed flowers.

Variables: $[i, j]$ = closed range over day counts · $k2$ = midpoint day · $\text{condition}(k2) = \text{canMake}(\text{day}=k2)$ yields $\geq m$ bouquets; MINIMIZE $\rightarrow$ answer = i **Pseudocode:**

```

if m*k > number of flowers: impossible, return -1
i = min(bloomDay), j = max(bloomDay)           # search the day axis
while i < j:
    k2 = midpoint
    if not canMake(k2): not enough days, condition false, go right (i = k2+1)
    else: k2 might be the answer (j = k2)
return i           # smallest feasible day count
# canMake(day): count adjacent runs of k bloomed flowers, true if bouquets >= m

```

```

class Solution {
    public int minDays(int[] bloomDay, int m, int k) {
        long need = (long) m * k;
        if (need > bloomDay.length) { // ← VARIATION: impossible if not enough fl
            return -1;
        }
        int i = Arrays.stream(bloomDay).min().getAsInt();
        int j = Arrays.stream(bloomDay).max().getAsInt(); // [i, j] over day counts
        // condition(k=k2): canMake(day=k2) yields >= m bouquets
        while (i < j) {
            int k2 = i + (j - i) / 2;
            if (!canMake(bloomDay, m, k, k2)) { // condition false → go right (not enough days)
                i = k2 + 1;
            } else { // condition true → k2 might be the answer
                j = k2;
            }
        }
        return i; // smallest feasible day count
    }
    private boolean canMake(int[] bloomDay, int m, int k, int day) {
        int bouquets = 0, adjacent = 0;
        for (int bloom : bloomDay) {
            if (bloom <= day) {
                adjacent++;
                if (adjacent == k) {
                    bouquets++;
                    adjacent = 0;
                }
            } else {
                adjacent = 0;
            }
        }
        return bouquets >= m;
    }
}

```

Time $O(n \log(\max(\text{bloomDay})))$ | **Space** $O(1)$

#1539 Kth Missing Positive Number

Description: Find the kth missing positive integer.

Template: Half-open $[i, j]$ — missing count before index k is $\text{arr}[k] - (k + 1)$, monotonic non-decreasing.

Intuition: At each index the number of missing positives so far is $\text{arr}[k] - (k+1)$; binary search the first index whose missing count is $\geq k$, then offset.

Variables: $[i, j]$ = half-open range over indices · mid = midpoint · $\text{condition}(\text{mid}) = \text{arr}[\text{mid}] - (\text{mid}+1) \geq k$ (missing count reaches k) · $\text{answer} = i + k$ **Pseudocode:**

```

i = 0, j = n
while i < j:
    mid = midpoint
    if arr[mid] - (mid+1) < k: too few missing, condition false, go right (i = mid+1)
    else: mid might be it (j = mid)
return i + k # i missing-free slots, then the kth missing

```

```

class Solution {
    public int findKthPositive(int[] arr, int k) {
        int i = 0, j = arr.length; // [i, j)
        // condition(k=mid): arr[mid] - (mid+1) >= k (missing count at mid reaches k)
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (arr[mid] - (mid + 1) < k) { // condition false → go right (too few missing)
                i = mid + 1;
            } else { // condition true → mid might be the answer
                j = mid;
            }
        }
        return i + k; // ← VARIATION: i missing-free slots + k
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#1818 Minimum Absolute Sum Difference

Description: Find the minimum absolute sum difference after one substitution.

Template: Half-open $[i, j)$ — for each element lowerBound a sorted copy of nums1 for the first value $\geq$ nums2[i], then compare it with the left neighbor.

Intuition: Total cost is the sum of absolute differences; one swap can only help where the closest available nums1 value beats the current difference. Find that closest value by binary search and track the best gain.

Variables: sorted = sorted copy of nums1 · total = sum of $|\text{nums1}[p] - \text{nums2}[p]|$ · maxGain = best reduction from one swap · $[i, j)$ = half-open range (per element) · k = midpoint · condition(k) = sorted[k] $\geq$ target (lowerBound for target = nums2[p]) **Pseudocode:**

```

sorted = sorted copy of nums1; total = 0; maxGain = 0
for each p:
    diff = |nums1[p] - nums2[p]|; total += diff; target = nums2[p]
    lowerBound first sorted value >= target over [0, n):
        if sorted[k] < target: go right (i = k+1) else j = k
    best = diff
    if i < n: best = min(best, |sorted[i] - target|) # value at/above target
    if i > 0: best = min(best, |sorted[i-1] - target|) # also check left neighbor
    maxGain = max(maxGain, diff - best)
return (total - maxGain) mod 1e9+7

```



```

class Solution {
    public int minAbsoluteSumDiff(int[] nums1, int[] nums2) {
        int MOD = 1_000_000_007;
        int n = nums1.length;
        int[] sorted = nums1.clone();
        Arrays.sort(sorted);
        long total = 0;
        int maxGain = 0;
        for (int p = 0; p < n; p++) {
            int diff = Math.abs(nums1[p] - nums2[p]);
            total += diff;
            int target = nums2[p];
            int i = 0, j = sorted.length; // [i, j): lowerBound first value >= target
            // condition(k): sorted[k] >= target
            while (i < j) {
                int k = i + (j - i) / 2;
                if (sorted[k] < target) { // condition false → go right
                    i = k + 1;
                } else { // condition true → k might be the answer
                    j = k;
                }
            }
            int best = diff;
            if (i < sorted.length) { // value at or above target
                best = Math.min(best, Math.abs(sorted[i] - target));
            }
            if (i > 0) { // ← VARIATION: also check left neighbor
                best = Math.min(best, Math.abs(sorted[i - 1] - target));
            }
            maxGain = Math.max(maxGain, diff - best);
        }
        return (int) ((total - maxGain) % MOD);
    }
}

```

Time $O(n \log n)$ | **Space** $O(n)$

#1891 Cutting Ribbons

Description: Find the maximum ribbon length such that m ribbons of that length can be cut.

Template: Answer space MAXIMIZE as "first infeasible" — half-open $[lo, hi+1)$, find the first length with $< k$ ribbons, then return $i - 1$ (no ceiling mid).

Intuition: Longer pieces produce fewer ribbons, so feasibility flips true→false as length grows. Search the first INFEASIBLE length ($condition(k) = countRibbons(k) < k$, false...false TRUE...true) and step back one; that is the largest feasible length. If no length is feasible the loop lands on lo , giving $i - 1 = 0$.

Variables: $[i, j)$ = half-open range over candidate lengths $[1, max+1)$ · mid = candidate length · $condition(mid) = countRibbons(mid) < k$ (first INFEASIBLE length); MAXIMIZE → answer = $i - 1$ **Pseudocode:**

```

max = longest ribbon
i = 1, j = max + 1      # half-open [1, max+1)
while i < j:
    mid = midpoint
    if countRibbons(mid) >= k: feasible, condition false, go right (i = mid+1)
    else: infeasible, mid might be the boundary (j = mid)
return i - 1           # last feasible length (0 if none)
# countRibbons(length): sum of (ribbon / length) over all ribbons

```

```

class Solution {
    public int maxLength(int[] ribbons, int k) {
        int max = 0;
        for (int ribbon : ribbons) {
            max = Math.max(max, ribbon);
        }
        int i = 1, j = max + 1; // ← VARIATION: [1, max+1) over lengths
        // condition(k=mid): countRibbons(mid) < k (mid is the first INFEASIBLE length) – MAXIMIZE
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (countRibbons(ribbons, mid) >= k) { // feasible → condition false → go right
                i = mid + 1;
            } else { // infeasible → condition true → mid might be it
                j = mid;
            }
        }
        return i - 1; // last feasible length (0 if none)
    }
    private long countRibbons(int[] ribbons, int length) {
        long count = 0;
        for (int ribbon : ribbons) {
            count += ribbon / length;
        }
        return count;
    }
}

```

Time $O(n \log(\max(\text{ribbons})))$ | **Space** $O(1)$